Universidad Distrital Francisco José de Caldas

Dept. of Computer Engineering

Systems Engineering

Workshop 4: Containerization, Acceptance, and CI/CD
Software Engineering Seminar

Carlos Andres Abella
Daniel Felipe Paez

Leidy Marcela Morales

Supervisor: Carlos Andrés Sierra

Bogotá D.C.

November 30, 2025

Contents

1 Introduction

This document presents the complete set of deliverables developed for Workshop 4 of the Software Engineering Seminar course. The primary objective of this workshop is to ensure that the Event Registration Platform is fully deployable, testable, and maintainable through the adoption of modern software engineering practices. To achieve this, the project integrates containerization, acceptance testing and continuous integration and continuous delivery (CI/CD) pipelines.

The platform components—Java backend, Python backend, and React frontend—were containerized using Docker to ensure consistency, isolation, and portability across environments. Acceptance testing was implemented through Apache Cucumber, allowing user requirements to be expressed in a human-readable format and automatically validated. Finally, a CI/CD workflow was implemented with GitHub Actions to automate testing, validation, and Docker image creation.

Altogether, these practices enhance the system's reliability, maintainability, and readiness for real-world deployment.

2 Dockerfiles and docker-compose.yml

This section describes how Docker was used to package, isolate, and orchestrate the different components of the Eventify platform. The objective of this approach is to provide a modular, portable, and easily deployable environment that supports both development and production workflows.

2.1 Service Containerization

Each component of the platform was containerized using a dedicated `Dockerfile`. The configuration of these files focuses on efficiency, portability, and clear separation of responsibilities.

2.1.1 Java Backend

The Java backend uses a multi-stage build strategy. The first stage compiles the Spring Boot application using Maven and downloads all dependencies, optimizing rebuild time. The final stage runs the generated JAR file inside a lightweight Temurin JRE image. The service exposes port 8081 and includes a health check that queries the `/health` endpoint to ensure the application's availability.

2.1.2 Java Backend Dockerfile

The Java backend uses a multi-stage build to separate the build environment from the runtime environment. This approach significantly reduces the final image size by only including the compiled JAR file and JRE in the runtime stage.

```
# Java Backend Dockerfile
FROM maven:3.9-eclipse-temurin-17 AS build

WORKDIR /app

# Copy pom.xml first for dependency caching
COPY pom.xml .
RUN mvn dependency:go-offline -B

# Copy source code
COPY src ./src

# Build the application
RUN mvn clean package -DskipTests

# Runtime stage
FROM eclipse-temurin:17-jre-alpine

WORKDIR /app

# Copy the built JAR from build stage
COPY --from=build /app/target/*.jar app.jar

# Expose port
EXPOSE 8081
```

```
26
27 # Health check
28 HEALTHCHECK --interval=30s --timeout=3s --start-period=40s --retries=3 \
29   CMD wget --no-verbose --tries=1 --spider http://localhost:8081/health || exit 1
30
31 # Run the application
32 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Listing 1: Java Backend Dockerfile

Key Features:

- Multi-stage build reduces final image size
- Dependency caching by copying `pom.xml` first
- Alpine-based runtime image for minimal size
- Health check endpoint configured for container orchestration
- Port 8081 exposed for external access

2.1.3 Python Backend

The Python backend is built using the minimal `python:3.12-slim` image. System packages such as `gcc` and `postgresql-client` are installed to support dependency compilation and database connectivity. The application's dependencies are installed from `requirements.txt`, and the service runs a FastAPI application through Uvicorn. It exposes port 8000 and implements a health check to monitor the service status.

2.1.4 Python Backend Dockerfile

The Python backend Dockerfile uses Python 3.12-slim as the base image and installs necessary system dependencies for PostgreSQL connectivity.

```
1  # Python Backend Dockerfile
2  FROM python:3.12-slim
3
4  WORKDIR /app
5
6  # Install system dependencies
7  RUN apt-get update && apt-get install -y \
8      gcc \
9      postgresql-client \
10     && rm -rf /var/lib/apt/lists/*
11
12 # Copy requirements file
13 COPY requirements.txt .
14
15 # Install Python dependencies
16 RUN pip install --no-cache-dir -r requirements.txt
17
18 # Copy application code
19 COPY . .
20
21 # Expose port
22 EXPOSE 8000
23
24 # Health check
25 HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
26   CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/
     ↪ health')" || exit 1
27
28 # Run the application
29 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Listing 2: Python Backend Dockerfile

Key Features:

- Slim Python image for reduced size

- PostgreSQL client included for database operations
- Health check using Python's urllib module
- Port 8000 exposed for API access

2.1.5 React Frontend

The React frontend is also built using a multi-stage approach. Node.js is used in the first stage to install dependencies and generate the production build, which is later served through an Nginx container. The service exposes port 80 and includes a custom Nginx configuration along with a health check on the root path.

2.1.6 React Frontend Dockerfile

The React frontend uses a multi-stage build: first building the application with Node.js, then serving it with Nginx in production mode.

```
# Multi-stage build for React Frontend
FROM node:20-alpine AS build

WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy source code
COPY . .

# Build the application
RUN npm run build

# Production stage with Nginx
FROM nginx:alpine

# Copy built files from build stage
COPY --from=build /app/build /usr/share/nginx/html

# Copy nginx configuration
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Expose port
EXPOSE 80

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost/ || exit 1

# Start nginx
CMD ["nginx", "-g", "daemon off;"]
```

Listing 3: React Frontend Dockerfile

Key Features:

- Multi-stage build separates build tools from production runtime
- Alpine-based images for minimal footprint
- Nginx serves static files efficiently
- Custom Nginx configuration for SPA routing
- Port 80 exposed (mapped to 3000 in docker-compose)

2.2 Orchestration with Docker Compose

The `docker-compose.yml` file coordinates all services required by the platform. It defines the backend services, frontend, and two database systems, each running in its own isolated container.

- **MySQL:** Used by the Java backend. It initializes with SQL scripts, exposes port 3307, and stores data in a persistent volume.
- **PostgreSQL:** Used by the Python backend. It exposes port 5433 and also uses a persistent volume for data durability.
- **Java Backend:** Connects to MySQL using environment variables and depends on the database health check.
- **Python Backend:** Connects to PostgreSQL and similarly depends on the readiness of the database.
- **React Frontend:** Served through Nginx and depends on both backend services.

All services share a custom bridge network (`eventplatform-network`), allowing secure and isolated communication. Sensitive values such as credentials, database URLs, and security tokens are handled through environment variables, improving portability and security. Each service is configured to restart automatically in case of failure.

2.2.1 Docker Compose Configuration

Docker Compose orchestrates all services, including databases, ensuring proper service dependencies and network isolation.

```
1
2
3 version: '3.8'
4
5 services:
6   # MySQL Database for Java Backend
7   mysql:
8     image: mysql:8.0
9     container_name: eventplatform-mysql
10    environment:
11      MYSQL_ROOT_PASSWORD: rootpassword
12      MYSQL_DATABASE: eventplatform_auth
13      MYSQL_USER: app_user
14      MYSQL_PASSWORD: AppStrongPass1!
15    ports:
16      - "3307:3306"
17    volumes:
18      - mysql_data:/var/lib/mysql
19      - ./java-backend/scripts/01-create-database.sql:/docker-entrypoint-initdb.d/01-
    ↪ create-database.sql:ro
20      - ./java-backend/scripts/02-seed-data.sql:/docker-entrypoint-initdb.d/02-seed-data
    ↪ .sql:ro
21    healthcheck:
22      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-
    ↪ prootpassword"]
23      interval: 10s
24      timeout: 5s
25      retries: 5
26    networks:
27      - eventplatform-network
28
29  # PostgreSQL Database for Python Backend
30  postgres:
31    image: postgres:15-alpine
32    container_name: eventplatform-postgres
33    environment:
34      POSTGRES_USER: postgres
35      POSTGRES_PASSWORD: postgres
36      POSTGRES_DB: eventplatform
37    ports:
38      - "5433:5432"
```

```
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./python-backend/scripts/01-create-database.sql:/docker-entrypoint-initdb.d/01-create-database.sql:ro
      - ./python-backend/scripts/02-setup-schema-and-data.sql:/docker-entrypoint-initdb.d/02-setup-schema-and-data.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - eventplatform-network

  # Java Backend (Spring Boot)
  java-backend:
    build:
      context: ./java-backend
      dockerfile: Dockerfile
    container_name: eventplatform-java-backend
    environment:
      SPRING_PROFILES_ACTIVE: docker
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/eventplatform_auth?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
      SPRING_DATASOURCE_USERNAME: app_user
      SPRING_DATASOURCE_PASSWORD: AppStrongPass1!
      JWT_SECRET: your-256-bit-secret-key-change-this-in-production-make-it-very-long-and-secure
      JWT_EXPIRATION: 86400000
      CORS_ALLOWED_ORIGINS: http://localhost:3000,http://localhost:80
      SERVER_PORT: 8081
    ports:
      - "8081:8081"
    depends_on:
      mysql:
        condition: service_healthy
    networks:
      - eventplatform-network
    restart: unless-stopped

  # Python Backend (FastAPI)
  python-backend:
    build:
      context: ./python-backend
      dockerfile: Dockerfile
    container_name: eventplatform-python-backend
    environment:
      POSTGRES_HOST: postgres
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: eventplatform
      DATABASE_URL: postgresql://postgres:postgres@postgres:5432/eventplatform
      JWT_SECRET: your-256-bit-secret-key-change-this-in-production-make-it-very-long-and-secure
      JWT_ALGORITHM: HS256
      CORS_ORIGINS: http://localhost:3000,http://localhost:80
    ports:
      - "8000:8000"
    depends_on:
      postgres:
        condition: service_healthy
    networks:
      - eventplatform-network
    restart: unless-stopped

  # React Frontend (Nginx)
  react-frontend:
    build:
      context: ./react-frontend
      dockerfile: Dockerfile
    container_name: eventplatform-react-frontend
    ports:
      - "3000:80"
```

```
    depends_on:
      - java-backend
      - python-backend
    networks:
      - eventplatform-network
    restart: unless-stopped

volumes:
  mysql_data:
    driver: local
  postgres_data:
    driver: local

networks:
  eventplatform-network:
    driver: bridge
```

Listing 4: docker-compose.yml

Orchestration Features:

- **Service Dependencies:** Backend services wait for database health checks
- **Network Isolation:** All services communicate through a dedicated bridge network
- **Volume Persistence:** Database data persists across container restarts
- **Health Checks:** Databases include health checks to ensure readiness
- **Automatic Restart:** Services automatically restart on failure
- **Port Mapping:** External ports mapped to avoid conflicts (3307, 5433)

2.2.2 Containerization Benefits

The containerization approach provides several advantages:

1. **Consistency:** Same environment across all stages of development
2. **Isolation:** Services are isolated and cannot interfere with each other
3. **Scalability:** Easy to scale individual services independently
4. **Portability:** Application runs identically on any Docker-compatible system
5. **Resource Efficiency:** Containers share the host OS kernel
6. **Easy Deployment:** Single command deploys entire application stack

Overall, containerization and orchestration significantly improve the maintainability, consistency, and deployment workflow of the Eventify platform.

3 Cucumber Feature Files and Test Results

To validate the system's behavior from a user-centric perspective, acceptance tests were implemented using **Apache Cucumber**. These tests define system requirements in a human-readable format (Gherkin) and verify them against the actual backend implementation.

3.1 Project Structure and Configuration

The acceptance testing infrastructure was established within a dedicated test module (‘Workshop-3/tests/java-backend‘) to maintain a clean separation of concerns. The following components were added:

- **Configuration:** The ‘pom.xml‘ was updated with ‘cucumber-java‘, ‘cucumber-spring‘, and ‘cucumber-junit-platform-engine‘.
- **Feature Files:** Located in ‘src/test/resources/features/‘, describing scenarios like ‘authentication.feature‘.

- **Step Definitions:** Java classes (e.g., ‘AuthStepDefinitions.java‘) that map Gherkin steps to executable code using ‘MockMvc‘.
- **Test Runner:** The ‘RunCucumberAcceptanceTest.java‘ class wires the JUnit Platform suite engine for execution.

3.2 Feature Coverage

The acceptance tests focus on critical workflows. For example, the authentication feature includes:

1. **Buyer Registration:** Validates that a new buyer can register, receiving a ‘201 Created‘ status.
2. **User Login:** Verifies that an existing user can log in and receive an authentication token.

3.3 Technical Implementation

The execution logic in ‘AuthStepDefinitions.java‘ leverages Spring’s ‘MockMvc‘ to simulate interactions:

- **Request Construction:** Builds DTOs (‘RegisterRequest‘, ‘LoginRequest‘) using existing app structures.
- **Validation:** Reuses regex patterns and validation rules to ensure consistency with production.
- **Assertions:** Asserts HTTP response codes and JSON payloads without modifying controller logic.

3.4 Test Results

The tests are integrated into the Maven build lifecycle and can be run via:

```
mvn clean test -Dtest=RunCucumberAcceptanceTest
```

Below is the evidence of the successful execution of the Cucumber scenarios:

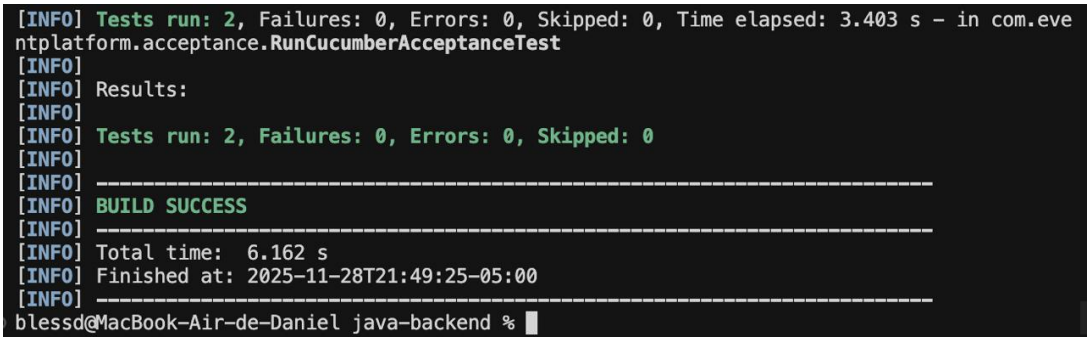

Figure 1: Execution log showing successful Cucumber acceptance tests.

4 GitHub Actions Workflow

The CI/CD pipeline implemented for the Event Registration Platform ensures continuous validation, automated testing, and reliable build processes across all system components. The workflow is defined using GitHub Actions, by incorporating unit tests, acceptance tests, backend coverage analysis, linting, and Docker image creation.

The pipeline is triggered on every push and pull request to the main and develop branches, ensuring early detection of defects and guaranteeing that only validated code is integrated into the main development flow. This automated process strengthens project maintainability, supports team collaboration, and enforces quality checks consistently.

```yaml
name: CI/CD Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]

env:
  JAVA_VERSION: '17'
  PYTHON_VERSION: '3.12'
  NODE_VERSION: '20'

jobs:
  test-java-backend:
    name: Test Java Backend
    runs-on: ubuntu-latest
    services:
      mysql:
        image: mysql:8.0
        env:
          MYSQL_ROOT_PASSWORD: rootpassword
          MYSQL_DATABASE: eventplatform_auth
        ports:
          - 3306:3306
        options: >-
          --health-cmd="mysqladmin ping"
          --health-interval=10s
          --health-timeout=5s
          --health-retries=5

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up JDK ${{ env.JAVA_VERSION }}
        uses: actions/setup-java@v3
        with:
          java-version: ${{ env.JAVA_VERSION }}
          distribution: 'temurin'
          cache: 'maven'

      - name: Run unit tests
        working-directory: ./Workshop-3/java-backend
        run: mvn clean test

      - name: Run acceptance tests
        working-directory: ./Workshop-3/tests/java-backend
        run: mvn clean test

      - name: Publish test results
        uses: EnricoMi/publish-unit-test-result-action@v2
        if: always()
        with:
          files: |
            Workshop-3/tests/java-backend/target/surefire-reports/*.xml

  test-python-backend:
    name: Test Python Backend
    runs-on: ubuntu-latest
```

```
    services:
      postgres:
        image: postgres:15-alpine
        env:
          POSTGRES_USER: postgres
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: eventplatform
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Python ${{ env.PYTHON_VERSION }}
        uses: actions/setup-python@v4
        with:
          python-version: ${{ env.PYTHON_VERSION }}
          cache: 'pip'

      - name: Install dependencies
        working-directory: ./Workshop-3/python-backend
        run: |
          pip install pytest pytest-cov

      - name: Run tests
        working-directory: ./Workshop-3/python-backend
        env:
          DATABASE_URL: postgresql://postgres:postgres@localhost:5432/eventplatform
        run: pytest tests/ -v --cov=app --cov-report=xml

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        if: always()
        with:
          file: ./Workshop-3/python-backend/coverage.xml
          flags: python-backend

  build-docker-images:
    name: Build Docker Images
    runs-on: ubuntu-latest
    needs: [test-java-backend, test-python-backend]

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v2

      - name: Build Java Backend image
        uses: docker/build-push-action@v4
        with:
          context: ./Workshop-3/java-backend
          file: ./Workshop-3/java-backend/Dockerfile
          push: false
          tags: event-platform/java-backend:latest
          cache-from: type=registry,ref=event-platform/java-backend:latest
          cache-to: type=inline

      - name: Build Python Backend image
        uses: docker/build-push-action@v4
        with:
          context: ./Workshop-3/python-backend
          file: ./Workshop-3/python-backend/Dockerfile
          push: false
          tags: event-platform/python-backend:latest
          cache-from: type=registry,ref=event-platform/python-backend:latest
```

```
          cache-to: type=inline

      - name: Build React Frontend image
        uses: docker/build-push-action@v4
        with:
          context: ./Workshop-3/react-frontend
          file: ./Workshop-3/react-frontend/Dockerfile
          push: false
          tags: event-platform/react-frontend:latest
          cache-from: type=registry,ref=event-platform/react-frontend:latest
          cache-to: type=inline

      - name: Test docker-compose
        working-directory: ./Workshop-3
        run: |
          docker-compose config
          docker-compose build

  lint-frontend:
    name: Lint React Frontend
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Node.js ${{ env.NODE_VERSION }}
        uses: actions/setup-node@v3
        with:
          node-version: ${{ env.NODE_VERSION }}
          cache: 'npm'
          cache-dependency-path: Workshop-3/react-frontend/package-lock.json

      - name: Install dependencies
        working-directory: ./Workshop-3/react-frontend
        run: npm ci

      - name: Run linter
        working-directory: ./Workshop-3/react-frontend
        run: npm run lint || true

      - name: Check build
        working-directory: ./Workshop-3/react-frontend
        run: npm run build
```

Listing 5: GitHub Actions CI/CD Workflow

4.1 Workflow Stages

The workflow file (‘.github/workflows/main.yml‘) defines steps for:

- Checking out the code.
- Setting up JDK and Python environments.
- Running Unit Tests (Maven/Pytest).
- Building Docker images.

4.1.1 1. Test Java Backend

- Sets up MySQL service container
- Runs unit tests using Maven
- Executes Cucumber acceptance tests
- Publishes test results

4.1.2 2. Test Python Backend

- Sets up PostgreSQL service container
- Installs Python dependencies
- Runs pytest with coverage reporting
- Uploads coverage reports to codecov

4.1.3 3. Build Docker Images

- Runs only after tests pass
- Builds all three Docker images
- Uses Docker layer caching for faster builds
- Validates docker-compose configuration

4.1.4 4. Lint Frontend

- Installs Node.js dependencies
- Runs ESLint (if configured)
- Validates that the frontend builds successfully